

Создание собственных инструментов: Автоматизация для пентестера и администратора на Python и PowerShell

 redsec.by/article/sozdanie-sobstvennyh-instrumentov-avtomatizaciya-dlya-pentestera-i-administratora-na-python-i-powershell

Red Teams | 13 ноября 2025



В мире кибербезопасности грань между рутинной работой и эффективным анализом угроз определяет один ключевой навык — автоматизация. Специалист, который тратит часы на ручную проверку логов или сканирование сетей, всегда будет на шаг позади того, кто делегировал эти задачи скриптам. Эта статья — ваш путеводитель в мир создания собственных инструментов. Мы рассмотрим, как с помощью Python и

PowerShell превратить повторяющиеся задачи в отлаженные автоматические процессы, высвобождая время для самого важного — принятия решений.

Почему автоматизация — это ваш лучший профессиональный актив

Представьте типичное утро системного администратора: необходимо вручную проверить 50 серверов на наличие критических обновлений, что занимает два часа драгоценного времени. Или вы — пентестер, который часами вручную ищет поддомены, рискуя упустить неочевидную, но критически важную точку входа. Эти сценарии — прямой путь к выгоранию и, что хуже, к ошибкам.

Автоматизация через скрипты решает эти проблемы фундаментально. Скрипт — это не просто код, это ваш цифровой ассистент, который работает без усталости и с абсолютной точностью. Один раз определив последовательность действий, вы можете воспроизводить её бесконечно, получая стабильный и предсказуемый результат. Вместо ингредиентов у нас IP-адреса, домены и системные логи, а вместо кухонной плиты — интерпретаторы Python и PowerShell.

Три столпа эффективности автоматизации:

- **Экономия времени:** Задачи, ранее занимавшие часы, выполняются за минуты или даже секунды.
- **Снижение человеческого фактора:** Скрипты выполняют команды точно по алгоритму, они не устают, не отвлекаются и не делают опечаток.
- **Масштабируемость:** Один и тот же скрипт может быть применён к сотням и тысячам целей одновременно, будь то серверы, рабочие станции или веб-приложения.

Концепции в простых аналогиях

Аналогия 1: Скрипты как швейцарский нож

Стандартное программное обеспечение — это как набор узкоспециализированных кухонных приборов: блендер делает только смузи, а тостер поджаривает хлеб. Скрипты же — это ваш персональный швейцарский нож: компактный, многофункциональный инструмент, который вы настраиваете под конкретную задачу. Нужно просканировать порты? Написали функцию. Собрать логи? Добавили еще одну. Проверить SSL-сертификаты? Легко!

Аналогия 2: Red Team и Blue Team — это хоккейный матч

Для тех, кто не погружен в кибербезопасность, концепции атакующих (Red Team) и защитников (Blue Team) могут показаться сложными. Представим хоккейный матч:

- **Red Team (Атакующие):** Это нападающие. Их цель — забить гол, то есть найти уязвимость и проникнуть в систему. Они изучают оборону соперника, ищут слабые места и используют любые разрешенные тактики для прорыва.
- **Blue Team (Защитники):** Это защитники и вратарь. Они блокируют атаки, отслеживают каждое движение противника и укрепляют оборону там, где обнаруживают бреши. Они постоянно анализируют игру, чтобы предвидеть следующий ход нападающих.
- **Purple Team (Тренерский штаб):** Это тренеры и аналитики, которые изучают игру обеих команд. Они помогают нападающим улучшить тактику, а защитникам — усилить оборону. В результате этого взаимодействия выигрывает вся организация, так как её общая защищенность растёт.

Аналогия 3: Автоматизация — это промышленный конвейер

Вспомните, как раньше автомобили собирали вручную — это был долгий, трудоемкий процесс с высоким процентом брака. С появлением конвейера каждый этап был стандартизирован и автоматизирован. Процесс стал повторяемым, быстрым и надежным. Ваши скрипты — это точно такой же конвейер. Один раз настроив процесс сбора логов или проверки обновлений, вы получаете его выполнение в автоматическом режиме, без сбоев и вашего прямого участия.

Python для Red Team: Создаем арсенал для атаки

Python де-факто стал языком номер один для специалистов по наступательной безопасности. Причина проста: низкий порог вхождения, колоссальное количество готовых библиотек и огромное сообщество. Рассмотрим два практических примера.

Пример 1: Сканер поддоменов на Python

Стратегическая важность: На этапе разведки (reconnaissance) ключевая задача — составить карту атакуемой инфраструктуры. Поддомены (например, dev.example.com или backup.example.com) часто остаются без должного внимания, на них может быть тестовое окружение со слабыми паролями или устаревшее ПО.

Простой скрипт на основе библиотеки requests:

```
import requests
import threading
from queue import Queue

# Целевой домен
domain = "example.com"

# Очередь для хранения поддоменов
q = Queue()
```

```

# Читаем список распространенных поддоменов из файла (например, из репозитория SecLi
with open("subdomains.txt", "r") as file:
    for line in file:
        q.put(line.strip())

# Обнаруженные поддомены
discovered = []
thread_count = 20 # Количество потоков

print(f"Сканирую {q.qsize()} возможных поддоменов для {domain}...\n")

def scan_subdomain():
    while not q.empty():
        subdomain = q.get()
        url_http = f"http://{subdomain}.{domain}"
        url_https = f"https://{subdomain}.{domain}"

        for url in [url_http, url_https]:
            try:
                response = requests.get(url, timeout=3, allow_redirects=True)
                # Проверяем, что ответ не является "пустым" или ошибкой
                if response.status_code < 400:
                    print(f"[+] Найден поддомен: {url} (Статус: {response.status_coc
                    discovered.append(url)
                    break # Если один протокол сработал, второй не проверяем
            except requests.ConnectionError:
                pass
            except requests.Timeout:
                pass
        q.task_done()

# Запуск потоков
for _ in range(thread_count):
    worker = threading.Thread(target=scan_subdomain)
    worker.daemon = True
    worker.start()

q.join() # Ждем завершения всех задач в очереди

# Сохранение результатов
with open("discovered_subdomains.txt", "w") as output:
    for item in discovered:
        output.write(item + "\n")

print(f"\nГотово! Найдено поддоменов: {len(discovered)}")

```

Как это работает:

1. Скрипт читает список потенциальных имен поддоменов из файла. Качественные списки можно найти в репозиториях вроде Seclists на GitHub.

2. Для ускорения процесса используется многопоточность (`threading`): несколько проверок выполняются параллельно.
3. Для каждого имени формируется URL по протоколам HTTP и HTTPS.
4. Отправляется HTTP-запрос. Если сервер успешно отвечает (код состояния меньше 400), поддомен считается существующим.
5. Все найденные поддомены сохраняются в файл для дальнейшего анализа.

Профессиональные улучшения:

- NS-запросы: Интегрируйте библиотеку `dnspython` для прямых DNS-запросов (тип A, CNAME), что часто быстрее и надежнее HTTP-проверок.
- Анализ Wildcard DNS: Добавьте проверку на wildcard-записи (*.example.com), которые могут давать ложноположительные результаты.
- Виртуальные хосты: Расширьте скрипт для поиска виртуальных хостов на найденных IP-адресах.

Пример 2: Базовый сканер портов на Python

Стратегическая важность: Открытые порты — это двери в систему. Зная, какие службы (SSH, FTP, HTTP) активны, пентестер может целенаправленно искать для них известные уязвимости.

Скрипт с использованием библиотеки `socket`:

```
python
import socket
from datetime import datetime
from concurrent.futures import ThreadPoolExecutor

# Цель сканирования
target = input("Введите IP или домен для сканирования: ")
try:
    target_ip = socket.gethostbyname(target)
except socket.gaierror:
    print("Ошибка: Не удалось разрешить имя хоста.")
    exit()

print(f"\nНачинаю сканирование {target} ({target_ip})")
print(f"Время начала: {datetime.now()}\n")
print("-" * 50)

# Функция проверки одного порта
def scan_port(port):
    try:
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
            sock.settimeout(1)
```

```

        result = sock.connect_ex((target_ip, port))
        if result == 0:
            print(f"[+] Порт {port} ОТКРЫТ")
            return port
    except socket.error:
        pass # Игнорируем ошибки подключения
    return None

# Сканирование наиболее распространенных портов с использованием пула потоков
common_ports = [21, 22, 23, 25, 53, 80, 110, 135, 139, 443, 445, 3306, 3389, 5900, 8
open_ports = []

with ThreadPoolExecutor(max_workers=100) as executor:
    results = executor.map(scan_port, common_ports)
    for port in results:
        if port:
            open_ports.append(port)

print("-" * 50)
print(f"Сканирование завершено: {datetime.now()}")
print(f"Открытые порты: {open_ports}")

```

Как это работает:

- Модуль `socket` используется для установления TCP-соединения с указанным портом.
- Метод `connect_ex()` не генерирует исключение при неудачном соединении, а возвращает код ошибки. Если он возвращает 0, соединение успешно, и порт считается открытым.
- `ThreadPoolExecutor` позволяет выполнять сканирование множества портов параллельно, значительно ускоряя процесс.
- Скрипт перебирает список наиболее часто используемых портов и выводит только открытые.

Профессиональные улучшения:

- **Использование `python-nmap`:** Для серьезной работы используйте библиотеку `python-nmap`, которая является оберткой для мощного сканера Nmap. Это позволит определять версии служб (banner grabbing), операционную систему и выполнять сложные NSE-скрипты.
SYN-сканирование: Для более "тихого" сканирования изучите библиотеку `Scapy`, которая позволяет создавать сетевые пакеты на низком уровне и выполнять SYN-сканирование (полуоткрытое соединение), которое реже логируется.
- **Асинхронность:** Для сканирования тысяч портов перепишите скрипт с использованием `asyncio` для более эффективного управления большим количеством одновременных соединений.

PowerShell для Blue Team: Автоматизация защиты и мониторинга

PowerShell — это основной инструмент для администраторов и специалистов по защите в Windows-инфраструктуре. Его глубокая интеграция с Active Directory, WMI и системными журналами делает его незаменимым.

Пример 3: Автоматический сбор критических логов с рабочих станций

Стратегическая важность: В крупной доменной сети с сотнями компьютеров ручной просмотр логов невозможен. Централизованный сбор событий безопасности позволяет оперативно выявлять аномалии: попытки подбора пароля, создание новых учетных записей или повышение привилегий.

Скрипт PowerShell:

```
# Список компьютеров для мониторинга (получаем из Active Directory)
$Computers = Get-ADComputer -Filter {Enabled -eq $true} | Select-Object -ExpandPrope

# Период сбора логов (за последние 24 часа)
$StartTime = (Get-Date).AddHours(-24)

# Файл для сохранения результатов
$OutputFile = "C:\Reports\SecurityLogs_$(Get-Date -Format 'yyyyMMdd').csv"

# Массив для хранения результатов
$Results = @()

Write-Host "Сбор логов безопасности с $($Computers.Count) компьютеров..." -Foregrou

foreach ($Computer in $Computers) {
    if (Test-Connection -ComputerName $Computer -Count 1 -Quiet) {
        try {
            # Получаем критические события
            $Events = Get-WinEvent -ComputerName $Computer -FilterHashtable @{
                LogName = 'Security'
                StartTime = $StartTime
                ID = 4625, 4672, 4720 # Неудачный вход, админ-права, создание учётк
            } -ErrorAction Stop

            foreach ($Event in $Events) {
                $Results += [PSCustomObject]@{
                    Computer    = $Computer
                    Time         = $Event.TimeCreated
                    EventID      = $Event.Id
                    Message      = $Event.Message.Split([Environment]::NewLine)[0] # E
                }
            }
            Write-Host "[OK] $Computer - найдено событий: $($Events.Count)" -Foregrc
        } catch {
            Write-Host "[ОШИБКА] $Computer - не удалось получить логи: $($_.Exceptic
        }
    } else {
```

```

        Write-Host "[ПРЕДУПРЕЖДЕНИЕ] $Computer недоступен" -ForegroundColor Yellow
    }
}

# Экспорт в CSV для дальнейшего анализа в Excel или SIEM
$Results | Export-Csv -Path $OutputFile -NoTypeInfo -Encoding UTF8

Write-Host "`nГотово! Результаты сохранены в $OutputFile" -ForegroundColor Yellow
Write-Host "Всего критических событий собрано: $($Results.Count)" -ForegroundColor Yellow

```

Как это работает:

- Командлет Get-ADComputer получает список всех активных компьютеров из Active Directory.
- Get-WinEvent удаленно подключается к журналу Security каждого компьютера. События фильтруются по ID: 4625 (неудачный вход в систему), 4672 (вход с правами администратора), 4720 (создание новой учетной записи пользователя).
- Все найденные события агрегируются в единый CSV-файл для централизованного анализа.

Планирование и интеграция: Настройте этот скрипт на ежедневный запуск через **Планировщик заданий Windows (Task Scheduler)**. Результаты можно автоматически отправлять в SIEM-систему или на почту дежурному администратору.

Пример 4: Проверка наличия критических обновлений на всех серверах

Стратегическая важность: Неустановленные обновления безопасности — одна из главных причин компрометации (вспомните эпидемии WannaCry и Petya, эксплуатировавшие уязвимость MS17-010). Автоматическая проверка позволяет поддерживать инфраструктуру в защищенном состоянии.

Скрипт PowerShell с использованием модуля PSWindowsUpdate:

```

# Шаг 1: Установка модуля (выполняется один раз)
# Install-Module -Name PSWindowsUpdate -Force -Scope AllUsers

# Импорт модуля в текущую сессию
Import-Module PSWindowsUpdate

# Получаем все серверы из Active Directory
$Servers = Get-ADComputer -Filter {OperatingSystem -like "*Server*"} | Select-Object

# Список критических обновлений (KB), которые должны быть установлены
$CriticalKBs = @('KB5000802', 'KB5000808', 'KB5000809') # Пример, замените на актуал

$Report = @()

Write-Host "Проверка обновлений на $($Servers.Count) серверах..." -ForegroundColor C

```



```

foreach ($Server in $Servers) {
    if (Test-Connection -ComputerName $Server -Count 1 -Quiet) {
        Write-Host "Проверяю $Server..." -ForegroundColor Yellow
        try {
            # Получаем список установленных обновлений удаленно
            $InstalledUpdates = Invoke-Command -ComputerName $Server -ScriptBlock {
                Get-HotFix | Select-Object -ExpandProperty HotFixID
            } -ErrorAction Stop

            # Находим отсутствующие критические обновления
            $MissingKBs = $CriticalKBs | Where-Object { $_ -notin $InstalledUpdates

            $Status = if ($MissingKBs) { "ТРЕБУЕТСЯ ОБНОВЛЕНИЕ" } else { "Актуально"

            $Report += [PSCustomObject]@{
                Server      = $Server
                Status      = $Status
                MissingKBs   = ($MissingKBs -join ', ')
                CheckedTime = Get-Date
            }

            Write-Host " Статус: $Status" -ForegroundColor (if ($MissingKBs) { "Red"
        } catch {
            $Report += [PSCustomObject]@{ Server = $Server; Status = "ОШИБКА ПРОВЕРКИ"
            Write-Host " [ОШИБКА] Не удалось получить данные" -ForegroundColor Red
        }
    } else {
        Write-Host "$Server НЕДОСТУПЕН" -ForegroundColor Magenta
    }
}

# Экспорт отчета в CSV
$ReportPath = "C:\Reports\UpdateStatus_$(Get-Date -Format 'yyyyMMdd_HH:mm').csv"
$Report | Export-Csv -Path $ReportPath -NoTypeInformation -Encoding UTF8

Write-Host "`n===== ИТОГИ =====" -ForegroundColor Cyan
Write-Host "Отчёт сохранён: $ReportPath" -ForegroundColor Yellow

```

Как это работает:

1. Используется сторонний, но ставший стандартом модуль PSWindowsUpdate для работы с обновлениями.
2. Get-ADComputer с фильтром по операционной системе выбирает только серверы.
3. `Invoke-Command` позволяет удаленно выполнить команду `Get-HotFix` на каждом сервере для получения списка установленных патчей.
4. Скрипт сравнивает список установленных обновлений с вашим "золотым списком" критических KB.

5. Формируется наглядный отчет, который сразу показывает проблемные серверы.

Автоматизация: Запланируйте еженедельное выполнение этого скрипта и настройте отправку отчета по email, чтобы держать руку на пульсе состояния безопасности вашей инфраструктуры.

Практические советы по написанию профессиональных скриптов

1. Начинайте с малого. Не пытайтесь создать универсальный комбайн. Решите одну конкретную задачу, а затем постепенно расширяйте функционал.
2. Используйте логирование. Всегда записывайте ключевые действия и ошибки в лог-файл. Это бесценно при отладке.
3. Обрабатывайте исключения. Ваш скрипт не должен "падать" из-за того, что один сервер из ста недоступен. Используйте ``try-except`` в Python и ``try-catch`` в PowerShell.
4. Комментируйте свой код. Через месяц вы сами не вспомните, почему выбрали именно такой подход. Комментарии — это документация для вас и вашей команды.
5. Тестируйте в изолированной среде. Никогда не запускайте новый скрипт сразу в производственной среде. Используйте виртуальные машины для безопасного тестирования.
6. Используйте систему контроля версий (Git). Хранение скриптов в Git позволяет отслеживать изменения, откатываться к рабочим версиям и эффективно работать в команде.
7. Параметризируйте скрипты. Не "зашивайте" в код IP-адреса или пароли. Передавайте их как аргументы командной строки или читайте из конфигурационных файлов.

Этика и легальность

Ключевое напоминание: Все инструменты и техники, описанные для Red Team, должны применяться исключительно в рамках легальных тестирований на проникновение с письменного разрешения владельца системы.

Несанкционированное сканирование и попытки взлома являются уголовным преступлением. Для Blue Team — убедитесь, что ваши скрипты для сбора данных соответствуют внутренним политикам безопасности и не нарушают приватность сотрудников.

Автоматизация — это не просто тренд, а фундаментальный навык, который отделяет эффективного специалиста по кибербезопасности от вечно занятого. Начав с простых скриптов, подобных приведенным в этой статье, вы не только сэкономите

сотни часов рабочего времени, но и получите глубокое понимание того, как на самом деле работают системы и сети.

Вы создадите собственный, уникальный арсенал инструментов, идеально заточенный под ваши задачи. Помните: лучшие специалисты — это не те, кто больше всех работает руками, а те, кто умнее всех автоматизирует свою работу. Начните создавать свои инструменты уже сегодня.